



EL2805 Reinforcement Learning

Exercise Session 1

October 27th 2025

Division of Decision and Systems Control
School of Electrical Engineering
KTH Royal Institute of Technology

Instructions.

- These exercises are for your practice, and you do not need to submit solutions. We encourage you to work on them before the corresponding exercise session on Oct 31st. The materials for the 2nd exercise session will be available on Nov 3rd.
- In each exercise session, we will go through a selection of topics chosen by the teaching assistant and participating students.
- Homework and labs will be based on concepts covered in lectures and exercise sessions. These materials are excellent preparation for your graded assignments!
- There is no need to memorize equations - relevant materials will be provided during the exam. Focus on concepts and understanding!
- The materials cover a wide range of topics, so feel free to focus on what interests you most. However, to develop a well-rounded understanding of RL, it is important that you engage with both theoretical and practical exercises.
- We use green boxes to highlight established results and definitions, yellow for problems that require your solutions, and pink for high-level remarks. Exercises marked with a ★ are more challenging and intended for students seeking an extra challenge.
- We value your feedback on the materials provided. If you have any questions, concerns, or suggestions, do not hesitate to reach out to any of the TAs.

1 The power of greediness: Why Policy Iteration works?

"I never had a policy; I have just tried to do my very best each and every day."

– A. Lincoln

1.1 [read] Refreshing the basics

Policy Iteration is a fundamental algorithm in dynamic programming, primarily used as a **planning** algorithm, relying on known rewards r and transition probabilities P . In contrast, most algorithms in RL combine both **learning** and **planning**. In this chapter, we will explore why Policy Iteration converges to the optimal solution. This chapter builds on concepts from Lecture 1 (Markov Decision Processes), as well as Sutton & Barto's book [1].

Evaluating all possible policies can be overwhelming. For a finite state space $\mathcal{S}$ (with cardinality $|\mathcal{S}|$) and finite action space $\mathcal{A}$ (with cardinality $|\mathcal{A}|$), the number of deterministic policies is $|\mathcal{A}|^{|\mathcal{S}|}$ - growing exponentially with the number of states! Yet, dynamic programming methods such as Policy Iteration converge to the optimal policy with only polynomial dependence on $|\mathcal{S}|$.

In this discussion, the discount factor is denoted as γ (in contrast to λ in the lecture notes). To evaluate a policy's effectiveness, we define the value function:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t^\pi, a_t^\pi) | s_0^\pi = s \right] \quad (\text{Value function for policy } \pi)$$

For a finite state space $\mathcal{S}$ and a fixed policy π , the value function V^π is a vector of length $|\mathcal{S}|$. We compare two value functions by denoting $V^\pi \geq V^{\pi'}$ if $V^\pi(s) \geq V^{\pi'}(s)$ for all states $s \in \mathcal{S}$. The max-norm of V^π is defined as $\|V^\pi\|_\infty = \max_{s \in \mathcal{S}} |V^\pi(s)|$. If rewards $r(s, a) \in [0, 1]$ then:

$$\|V^\pi\|_\infty = \max_s |V^\pi(s)| = \max_s \left| \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t^\pi, a_t^\pi) | s_0^\pi = s \right] \right| \leq \sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma}.$$

Thus, the value function for any policy is bounded by $\frac{1}{1-\gamma}$.

Algorithm 1 Policy Iteration

- 1: Initialize policy π_0 , $k \leftarrow -1$
 - 2: **repeat**
 - 3: $k \leftarrow k + 1$
 - 4: **Policy evaluation:** compute V^{π_k}
 - 5: **Policy improvement:** $\pi_{k+1} \leftarrow \text{Greedy}(V^{\pi_k})$
 - 6: **until** $\forall s : \pi_{k+1}(s) = \pi_k(s)$
-

The Policy Iteration algorithm consists of two key phases: **policy evaluation** and **policy improvement**. During the policy evaluation phase, we estimate the value function of the current policy π_k . In the policy improvement phase, we derive a new policy π_{k+1} by selecting greedy actions based on V^{π_k} . Since V^{π_k} evaluates states rather than actions, we will also compute the state-action value function $Q^{\pi_k}(s, a)$:

$$Q^{\pi_k}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} V^{\pi_k}(s').$$

The greedy step $\pi_{k+1} \leftarrow \text{Greedy}(V^{\pi_k})$ then updates the policy as follows:

$$\forall s \in \mathcal{S} : \pi_{k+1}(s) = \underset{a}{\operatorname{argmax}} [r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} V^{\pi_k}(s')] \quad (\text{Policy improvement})$$

So far we have omitted the specifics of calculating V^{π_k} in the **policy evaluation** phase. For that, we are going to use the Bellman evaluation operator:

$$(\mathcal{L}_\pi V)(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s') \right] \quad (\text{Bellman evaluation operator})$$

For deterministic policies π , we can express this concisely in a vector form as $\mathcal{L}_\pi V = r_\pi + \gamma P_\pi V$ where $r_\pi(s) = r(s, \pi(s))$, $P_\pi(s, s') = P(s'|s, \pi(s))$. When $\pi = \pi^*$, the optimal policy, the operator $\mathcal{L}_{\pi^*}$ is denoted by $\mathcal{L}$ and called the Bellman optimality operator:

$$(\mathcal{L}V)(s) = \max_{a \in \mathcal{A}} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s') \right] \quad (\text{Bellman optimality operator})$$

The Bellman operators satisfy $V^\pi = \mathcal{L}_\pi V^\pi$ (please check §4.1 in [1] to confirm this). Solving the system of $|\mathcal{S}|$ equations of the form $V(s) = (\mathcal{L}_\pi V)(s)$ gives as one of the solutions V^π . In order to ensure this is the *only* solution, recall that Bellman operators are γ contractions:

$$\forall \pi : \quad \|\mathcal{L}_\pi V - \mathcal{L}_\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty, \quad (\text{Bellman op. are } \gamma\text{-contractions})$$

Here we can immediately see benefits of vector notation we introduced since we can quickly check that $\mathcal{L}_\pi V - \mathcal{L}_\pi V' = \gamma P_\pi (V - V')$ and the result follows immediately (recall that for transition matrices $\sum_{s' \in \mathcal{S}} P(s'|s, \pi(s)) = 1$).

Because of the contraction property, it follows (using Banach fixed-point theorem) that V^π is the unique solution of $V = \mathcal{L}_\pi V$. Thus, in the policy evaluation phase, for a fixed policy π_k , we compute V^{π_k} by solving the following set of $|\mathcal{S}|$ linear equations:

$$V^{\pi_k}(s) = \sum_{a \in \mathcal{A}} \pi_k(a|s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^{\pi_k}(s') \right] \quad (\text{Policy evaluation})$$

Now that we have derived all the steps of the PI algorithm, let's illustrate how it works in a simple example. In each iteration, the algorithm selects a policy greedily, resulting in deterministic policies (for ties, we can arbitrarily select one of the optimal actions). Figure 1 illustrates this for a setting with two states, $\mathcal{S} = \{s_1, s_2\}$. Each point within the green area represents the value vector $V^\pi(s)$ for some (possibly stochastic) policy π . The edges of the polytope correspond to value functions of deterministic policies, while the optimal value vector V^* is always located in the top right corner, since $V^*(s) \geq V^\pi(s)$ for all policies and states.

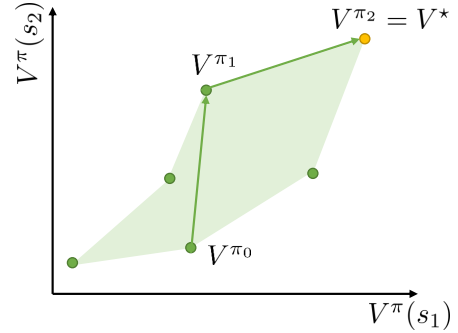


Figure 1: PI polytope

As we apply the PI algorithm, we move across different vertices of this polytope. Crucially, the strategy ensures that the value function increases in at least one coordinate while keeping the others constant. In other words, the PI algorithm provides an efficient way to explore the vertices of this polytope while monotonically improving performance. For more information about this concept and similar polytopes check [2].

1.2 [solve] Showing that Policy Iteration works

In this section, you will prove that Policy Iteration (PI) converges. For that we first need:

Exercise 1.1 [Policy Improvement Theorem] Let π be any policy and let $\pi' = \text{Greedy}(V^\pi)$ - in other words, let $\pi'(s) = \arg\max_a Q^\pi(s, a)$. Show that for every state s :

$$V^\pi(s) \stackrel{(1)}{\leq} Q^\pi(s, \pi'(s)) \stackrel{(2)}{\leq} V^{\pi'}(s)$$

△Hints

- (1) Write V^π in terms of Q^π and use that $\pi'(s) = \arg\max_a Q^\pi(s, a)$.
- (2) Write Q^π in terms of V^π and apply (1) recursively.

In words, this theorem states that if we choose a policy greedily with respect to the value function of the previous policy, then the new policy will perform better than (or equal to) the previous one. Now you are ready to show the following result:

Exercise 1.2 [*Convergence of PI*] Show that the PI algorithm converges to an ε -optimal policy after at most $k = \log\left(\frac{1}{\varepsilon(1-\gamma)}\right) / \log\left(\frac{1}{\gamma}\right)$ iterations.

△Hints

1. First rewrite inequality (2) from Exercise 1.1 as $\mathcal{L}V^{\pi_{k-1}} \leq V^{\pi_k}$. Then using that inequality and the contraction property, show that:

$$\forall k : \quad \|V^* - V^{\pi_k}\|_{\infty} \leq \gamma \|V^* - V^{\pi_{k-1}}\|_{\infty}.$$

2. Combine k of these inequalities and use the upper bound for $\|V\|_{\infty}$.

Before proceeding further, reflect on the properties used to prove convergence!

1.3 [code] On thin ice: Coding Policy Iteration

In this exercise, you will train an agent to navigate a frozen lake. An initial code template is provided in the `PI.FrozenLake.py` script.

Your tasks are as follows:

1. Familiarize yourself with the environment by reading:
`gymnasium.farama.org/environments/toy_text/frozen_lake/`
Set up the environment and explore the structure of `env.P`. How does `env.P` change when `is_slippery=True`? Why? Next complete the missing (TODO) lines in the script.
2. Determine by hand the optimal value of each state in the basic environment. Then plot the changes in $V^{\pi_k}(s) - V^{\pi^*}(s)$ for each k and s - why do the states group into six groups? Can you identify which state is being updated at which iteration? Additionally, plot how $\|V^{\pi_k} - V^{\pi^*}\|_{\infty}$ evolves with k . Is this curve always monotonically decreasing? Why (not)?
3. Set the `is_slippery=True` and analyze how stochasticity affects both the number of iterations required and the resulting policy. How does the length of the path change? Run multiple episodes with your trained policy and create a histogram of the path lengths. Is the number of path that reach the maximum episode length equal to the number of unsuccessful episodes? Why (not)?
4. Deterministic policies can be too strict. How would you implement a stochastic policy? What would change if policies were defined as a softmax of Q-values instead of greedy selection?
5. Explore the largest environment you can solve in a reasonable amount of time. Experiment with creating your own, more challenging environments that require additional iterations.



References

- [1] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [2] Robert Dadashi, Adrien Ali Taiga, Nicolas Le Roux, Dale Schuurmans, and Marc G Bellemare. The value function polytope in reinforcement learning. In *International Conference on Machine Learning*, pages 1486–1495. PMLR, 2019.

2 Beware: Supervised Learning can fail in sequential tasks

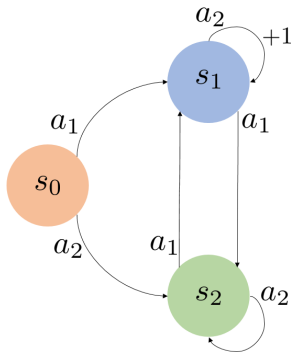
2.1 [read] Intro to Behavior Cloning

In this chapter, we aim to explore why reinforcement learning is necessary compared to other machine learning paradigms, such as supervised learning. Imagine you have access to an expert who can solve a task - for example, a driver navigating a car through traffic. You record what the expert does in various situations: which actions they take in which states. This gives you a dataset of state-action pairs $((s_1, a_1), (s_2, a_2), \dots)$ sampled from the expert's behavior.

The goal of behavior cloning is to learn a policy that imitates the expert. We do this by treating it as a supervised learning problem: the state is the input, the expert's action is the label, and we train a model to predict the expert's action from each state. Because the agent only learns from this fixed dataset and never observes the consequences of its own actions during training, there is no reward signal and no trial-and-error learning, which distinguishes behavior cloning from reinforcement learning.

In practice, behavior cloning works well when the agent only needs to act in states similar to those seen in the expert data. However, if the agent makes a mistake and reaches a state not present in the dataset, it may fail to recover - an issue we will explore next.

2.2 [solve] A simple MDP where Supervised Learning fails



Let $\mathcal{S} = \{s_0, s_1, s_2\}$ be the state space, with s_0 as the fixed initial state. For each state, we have actions $\mathcal{A} = \{a_1, a_2\}$. The discount factor is γ , and rewards are defined as $r(s, a) = 0$ for all s, a except $r(s_1, a_2) = 1$. Transitions are deterministic (see Figure 2).

As a warm-up, consider the following:

Exercise 2.1 Determine π^* for this MDP:

$\pi^*(s_0) = \underline{\hspace{1cm}}$, $\pi^*(s_1) = \underline{\hspace{1cm}}$, $\pi^*(s_2) = \underline{\hspace{1cm}}$

Next, we introduce the concept of the discounted occupancy measure for a policy π . This is a $|\mathcal{S}|$ -dimensional vector that quantifies the likelihood of being in a particular state under policy π :

Figure 2: A 3-state MDP

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t^\pi = s)$$

Verify that it forms indeed a probability distribution ($\sum_s d^\pi(s) = 1$). We use d^* to denote d^{π^*} .

Exercise 2.2 Recall that s_0 is the fixed initial state. Determine d^* :

$d^*(s_0) = \underline{\hspace{1cm}}$, $d^*(s_1) = \underline{\hspace{1cm}}$, $d^*(s_2) = \underline{\hspace{1cm}}$.

You are given an expert dataset $(s_1, a_1, s_2, a_2, \dots)$ where the states s are sampled from the expert's discounted state distribution d^* , and the actions are those taken by the expert policy π^* . Your task is to learn a policy π that mimics the expert. Importantly you do not have access to π^* - only to its behavior in the form of these state-action pairs. After running your supervised learning algorithm, you learn the following policy:

$$\hat{\pi}(s_0) = \begin{cases} a_1, & \text{with probability } 1 - \frac{\varepsilon}{1-\gamma} \\ a_2, & \text{with probability } \frac{\varepsilon}{1-\gamma} \end{cases} \quad \hat{\pi}(s_1) = a_2, \quad \hat{\pi}(s_2) = a_2$$

Exercise 2.3 Show that policy $\hat{\pi}$ fits the expert data almost perfectly:

$$\mathbb{P}_{s \sim d^*}(\hat{\pi}(s) \neq \pi^*(s)) = \varepsilon$$

At first glance, your learned policy $\hat{\pi}$ predicts the expert's actions accurately on the states in the dataset and seems successful. However, when you deploy $\hat{\pi}$, it interacts with the environment and generates its own trajectory of states, following the distribution $d^{\hat{\pi}}$ rather than d^* .

Exercise 2.4 First, calculate $d^{\hat{\pi}}$: $d^{\hat{\pi}}(s_0) = ______$, $d^{\hat{\pi}}(s_1) = ______$, $d^{\hat{\pi}}(s_2) = ______$. Then, show that under policy $\hat{\pi}$, the error is amplified as follows:

$$\mathbb{P}_{s \sim d^{\hat{\pi}}}(\hat{\pi}(s) \neq \pi^*(s)) = \frac{\varepsilon}{1 - \gamma}.$$

Because $\hat{\pi}$ may make mistakes it can visit states that the expert rarely (or never) visited. As a result, errors can accumulate: a small prediction error ε on the expert states can lead to a much larger error $\frac{\varepsilon}{1-\gamma}$ on the states encountered under $\hat{\pi}$. This phenomenon is known as **distribution shift** - the difference between the training distribution d^* and the testing or deployment distribution $d^{\hat{\pi}}$ - and is a fundamental reason why behavior cloning can fail in sequential tasks.

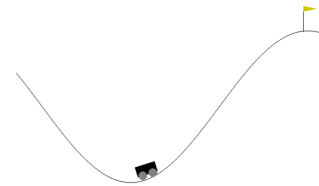
Why is the $\frac{1}{1-\gamma}$ factor important? While the error in the previous exercise is amplified by the factor $\frac{1}{1-\gamma}$, you might wonder how significant that increase is. To illustrate, consider a discounted MDP where we truncate the reward after H steps. The error can be bounded as follows:

$$\gamma^H(r_H + \gamma r_{H+1} + \dots) \leq \gamma^H \sum_{t=0}^{\infty} \gamma^t = \frac{\gamma^H}{1 - \gamma} \leq \frac{\exp(H(\gamma - 1))}{1 - \gamma}$$

where the last inequality follows from $1 + x \leq \exp(x)$. To ensure the approximation error is below a certain threshold, say δ , it suffices to choose $H = \frac{\log(\frac{\varepsilon}{1-\gamma})}{1-\gamma}$. Ignoring the numerator, we observe that $H \propto \frac{1}{1-\gamma}$. Hence, the effective horizon $\frac{1}{1-\gamma}$ corresponds to the trajectory length needed to maintain a small approximation error, similar to the horizon length in finite-horizon MDPs. This means that supervised learning error are multiplied by the effective horizon length - an undesirable characteristic in long-horizon problems.

2.3 [code] Hands-On: When Supervised Learning performs well ★

The theoretical results we have discussed represent worst-case scenarios. In certain situations, methods based on supervised learning can still perform well, especially when provided with high-quality expert data. To test this, take a look at the notebook titled `Imitation.MountainCar.ipynb` and run it on Google Colab <https://colab.research.google.com/>. You will run the code on the MountainCar environment to see if you can imitate an expert policy successfully. Instructions are provided in the notebook.



★ **Further reading.** Even though the results in this chapter may seem discouraging, significant progress has been made to address this problem. If you are interested in learning more, check out:

- DAgger algo: <https://imitation.readthedocs.io/en/latest/algorithms/dagger.html>
- Robotics application: <https://proceedings.mlr.press/v164/jang22a/jang22a.pdf>
- Imitation learning tutorial: <https://sites.google.com/view/icml2018-imitation-learning/>